

B28: SQL SuperHero Program : Session 31

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

8 April 2025 at 12:23

Exception Handling

An exception is an error condition during a program execution. PL/SQL supports programmers to catch such conditions using the EXCEPTION block in the program and an appropriate action is taken against the error condition.

To avoid such an error situation, we have to do exception handling. It prevents the program from giving errors and catching the exception.

System-defined exceptions --> whenever you can see ORA- then it is system defined exception

User-defined exceptions

Syntax for Exception Handling

The general syntax for exception handling is as follows. Here you can list down as many exceptions as you can handle. The default exception will be handled using WHEN others THEN –

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling goes here >
  WHEN exception1 THEN
    exception1-handling-statements
  WHEN exception2 THEN
    exception2-handling-statements
  WHEN exception3 THEN
    exception3-handling-statements
  .....
  WHEN others THEN
    exception3-handling-statements
END;
```

NO_DATA_FOUND

This exception will come, when the select statement does not find any record in the table against the where clause. It is the system defined exception.

```
declare
  l_region_name hr.regions.region_name%type;
Begin
  select region_name into l_region_name from hr.regions where region_id = 5;
  dbms_output.put_line(l_region_name);
End;
```

no data for
id=5

ORA-01403: no data found ORA-06512: at line 4
ORA-06512: at "SYS.DBMS_SQL", line 1721

→ This error occurred bcoz no data exist for id=5

More Details: <https://docs.oracle.com/error-help/db/ora-00100>

If you run this code in real SQL developer tool, this error will look like below:

```
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7     select Region_name Into Var_reg_name from regions where region_id = 5;
8     dbms_output.put_line(Var_reg_name);
9 End;
```

↓
No record

Query Result x Script Output x
Task completed in 1.069 seconds

Error starting at line : 4 in command -
Declare
Var_reg_name regions.Region_name%type;
Begin
select Region_name Into Var_reg_name from regions where region_id = 5;
dbms_output.put_line(Var_reg_name);
End;
Error report -
ORA-01403: no data found
ORA-06512: at line 4
01403. 00000 - "no data found"
*Cause: No data was found from the objects.
*Action: There was no data from the objects which may be due to end of fetch.

→ Because no data exist for region-id = 5

Now how to handle this:

```
2
3 set SERVEROUTPUT ON;
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7     select Region_name Into Var_reg_name from regions where region_id = 5;
8     dbms_output.put_line(Var_reg_name);
9     Exception when no_data_found then
10         Null;
11     dbms_output.put_line('Data not exists for this region_id');
12 End;
```

Data Not Exist for this region-id
↑
Control goto Exception block

Query Result x Script Output x
Task completed in 0.533 seconds

PL/SQL procedure successfully completed.
Data not exists for this region id

→ This is printed as written in Exception block.
No Error, becoz we handled the Exceptions.

Let me add one record in region table:

```
select * from regions where region_id = 1
```

REGION_ID	REGION_NAME
1	Europe
1	India

Normal SQL

2 rows selected.

```
2
3 set SERVEROUTPUT ON;
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7 select Region_name Into Var_reg_name from regions where region_id = 1;
8 dbms_output.put_line(Var_reg_name);
9 Exception when no_data_found then
10 Null;
11 dbms_output.put_line('Data not exists for this region_id');
12 End;
```

This statement gives 2 records and we have 1 variable only to hold.

Control goes to exception but nothing to handle this

2 value in 1 Column

Task completed in 1.077 seconds

```
dbms_output.put_line(Var_reg_name);
Exception when no_data_found then
Null;
dbms_output.put_line('Data not exists for this region_id');
End;
Error report -
ORA-01422: exact fetch returns more than requested number of rows
ORA-06512: at line 4
01422. 00000 - "exact fetch returns more than requested number of rows"
*Cause: The number specified in exact fetch is less than the rows returned.
*Action: Rewrite the query or change number of rows requested
```

Here we can see for region_id = 1, we have 2 records, but in our code we are taking this region_name into only one variable, that is var_reg_name.

So how can 1 variable hold 2 things???

1 can hold only one, so it gives error --> exact fetch returns more than requested number of rows

whenever we see this error, it means we have to do exception handling. and this time it is TOO_MANY_ROWS

TOO_MANY_ROWS

```

2
3 set SERVEROUTPUT ON;
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7   select Region_name Into Var_reg_name from regions where region_id = 1;
8   dbms_output.put_line(Var_reg_name);
9   Exception when no_data_found then
10     Null;
11   dbms_output.put_line('Data not exists for this region_id');
12   when too_many_rows then
13     dbms_output.put_line('more than 1 record exists');
14 End;
15
16

```

issue on #7, controller go to #9.

I cannot handle this sorry! then goto next

yes, I am expect

Handler

Query Result x Query Result 1 x Script Output x

Task completed in 1.262 seconds

PL/SQL procedure successfully completed.
more than 1 record exists

In the above example, we can see that 2 values are coming and can not be stored in 1 variable. controller will not execute line #8 and go to the exception block, line #9. Here we have exception handling started. But the first one when_no_data_found, can not handle this. so 10,11 will not execute and the controller goes to line #12. Here we can see when TOO_MANY_ROWS, yes, that is what the program is looking for. And it handles and no errors will come.

When OTHERS

```

3 set SERVEROUTPUT ON;
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7   select Region_name Into Var_reg_name from regions where region_id = 1;
8   dbms_output.put_line(Var_reg_name);
9   Exception when no_data_found then
10     Null;
11   dbms_output.put_line('Data not exists for this region_id');
12   when others then
13     dbms_output.put_line('sorry man, we have issues here');
14 End;
15
16

```

issue because more than 1 record exist

go to exception sorry. I can not

can handle anything

Prints

Query Result x Query Result 1 x Script Output x

Task completed in 0.518 seconds

PL/SQL procedure successfully completed.
sorry man, we have issues here

"When Others" IS the final exception and can handle any exception. If some particular exception is not handles and we have when others are defined, it will be handled here.

```
3 set SERVEROUTPUT ON;
4 Declare
5 Var_reg_name regions.Region_name%type;
6 Begin
7 select Region_name Into Var_reg_name from regions where region_id = 5;
8 dbms_output.put_line(Var_reg_name);
9 Exception when no_data_found then
10 Null;
11 dbms_output.put_line('Data not exists for this region_id');
12 when others then
13 dbms_output.put_line('sorry man, we have issues here');
14 End;
```

no data for this record
go to Exception
yes I can handle

Task completed in 0.523 seconds

PL/SQL procedure successfully completed.
Data not exists for this region_id

--> If exception occurs and we have already defined that exception explicitly, then controller will not go to when others, (does not matter that when others can handle anything, but here we have defined NO_DATA_FOUND and it goes there first and handles)

When Others will always be the last exception. If we try to define that exception before any other exception, it will compile time error.

The Program will not be compiled and give errors.

```
declare
    l_region_name hr.regions.region_name%type;
Begin
    select region_name into l_region_name from regions where region_id = 5;
    dbms_output.put_line(l_region_name);
Exception
    when others then
        dbms_output.put_line('Hello');
    when no_data_found then
        dbms_output.put_line('Hi');
End;
```

Program gives error.

when others, should be last exception always

ORA-06550: line 7, column 5:
PLS-00370: OTHERS handler must be last among the exception handlers of a block

```

declare
    l_region_name hr.regions.region_name%type;
Begin
    select region_name into l_region_name from regions where region_id = 5;
    dbms_output.put_line(l_region_name);
Exception
    when too_many_rows then
    dbms_output.put_line('Hello ji');
    when others then
        dbms_output.put_line('Hello');
    when no_data_found then
        dbms_output.put_line('Hi');
End;

```

→ Always be last among all

ORA-06550: line 9, column 5:
 PLS-00370: OTHERS handler must be last among the exception handlers of a block

] - error

```

declare
    l_region_name hr.regions.region_name%type;
Begin
    select region_name into l_region_name from regions where region_id = 5;
    dbms_output.put_line(l_region_name);
Exception
    when too_many_rows then
    dbms_output.put_line('Hello ji');
    when no_data_found then
        dbms_output.put_line('Hi');
        when others then
            dbms_output.put_line('Hello');
End;

Statement processed.
Hi

```

↓ No data

→ if any issue came, if exception presents, it will go there

=====

See the below example, a function created :

```

create or replace function get_sal(p_name varchar2) return number as
l_sal number;
Begin
    select salary into l_sal from hr.employees where first_name = p_name;
    return l_sal;
Exception
    when others then

```

```
dbms_output.put_line('so many records');
```

```
Return 1;
```

```
End get_sal;
```

```
create or replace function get_sal(p_name varchar2) return number as
l_sal number;
Begin
  select salary into l_sal from hr.employees where first_name = p_name;
  return l_sal;
Exception
  when others then
    dbms_output.put_line('so many records');
    Return 1;

End get_sal;

Function created.
```

more than 1 record

map to Exception

it can handle

cell here. for steven, we have 2 records, if go to Exception

GET_SAL('STEVEN')
1

so many records